

Git 和 GitHub 操作入门指南

本地版本管理核心工具 · 全球领先的云端软件开发协作平台



GitHub

从代码版本的本地追踪到团队的云端协同开发，掌握 Git 与 GitHub 的核心工作流，为高效、规范的软件开发打下坚实基础，开启现代化编程协作的全新篇章。

Author: XinYou

Date: 2026-05

Git 与 GitHub 的区别



Git: 本地版本管理工具

安装在电脑本地的软件，负责追踪文件的修改历史，就像一个强大的本地存档系统，无需联网即可工作。



GitHub: 远程代码托管平台

基于 Git 的云端服务，不仅是代码的云盘，更是开发者的协作社区，支持多人共同维护同一个项目。

Repository (仓库)

可以理解为一个项目文件夹，包含了项目的所有文件、历史记录和元数据。

Commit (提交)

就像游戏存档一样，是对项目当前状态的一次快照，记录了文件的修改内容。

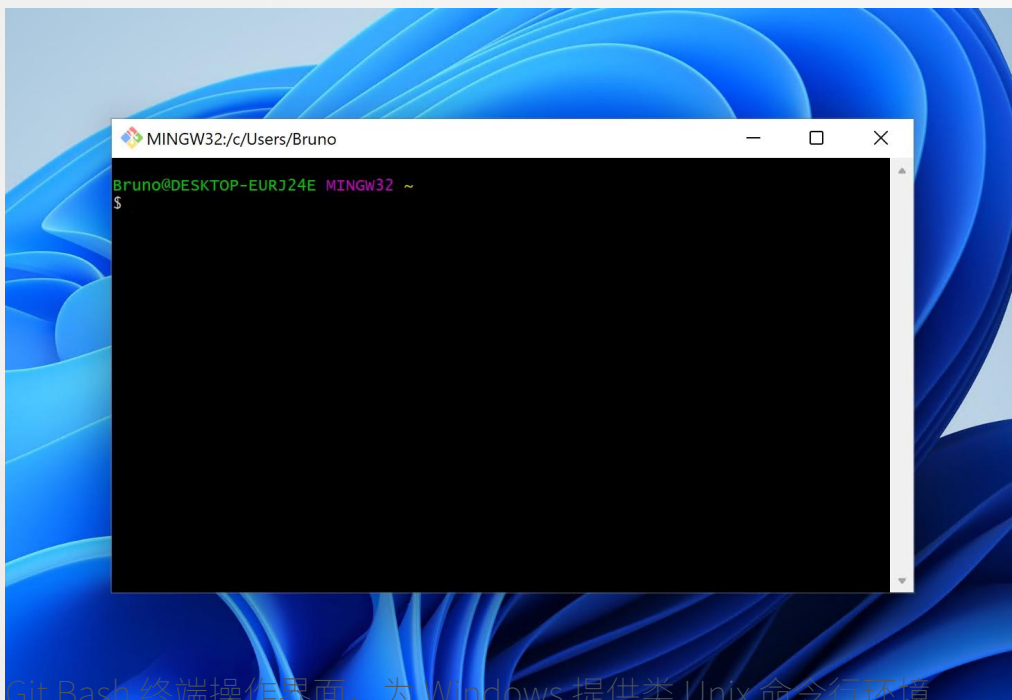
Branch (分支)

平行宇宙般的独立开发线，让你可以在不影响主版本的前提下，开发新功能或修复Bug。

协作与同步

Push上传、Pull拉取、Clone下载，通过这些指令实现本地与云端代码的同步与共享。

安装 Git 与初始配置



Git Bash 终端操作界面，为 Windows 提供类 Unix 命令行环境



验证安装结果：在终端输入 `git --version`，若输出 Git 的版本号，即表示安装成功。



Windows 系统安装

访问 git-scm.com/download/win 下载安装包。安装完成后，使用“Git Bash”工具来执行 Git 命令，它提供了更友好的命令行体验。



macOS 系统安装

推荐使用包管理器：`brew install git`。或在终端执行 `xcode-select --install`，通过 Xcode 命令行工具安装。



配置用户身份（必须）：执行 `git config --global` 命令设置用户名和邮箱，这是 Git 提交记录的身份标识。

Git 核心工作流程



01. 修改文件

在本地工作目录中自由编辑、新增或删除项目文件，这是开发工作的起点。



02. git add (放入暂存区)

将修改的文件“加入购物车”，标记为待提交状态，告诉 Git 这些改动是需要被记录的。



03. git commit

将暂存区的变更提交到本地版本库，生成不可变的版本快照，完成“本地存档”。



04. git push

将本地提交的版本推送到远程仓库（如 GitHub），实现代码的云端同步与团队共享。

“一句话理解 Git 流程”

改代码 → 准备（暂存）→ 存档（提交）→ 云端（推送）。这就像写文章时先打草稿，再定稿，最后发布到平台上分享的过程。

核心逻辑：Git 并不直接追踪文件变化，而是关注数据的整体快照，通过“暂存-提交-推送”的闭环确保版本的可追溯性。

从零创建 Git 项目



```
$ mkdir my-first-project && cd my-first-project  
$ echo "# My First Project" > README.md  
$ git init # 初始化本地Git仓库，生成 .git 目录  
$ git add README.md # 将文件添加到暂存区  
$ git commit -m "first commit" # 提交变更到本地仓库，记录版本  
$ git log --oneline # 查看简洁的提交历史记录
```

操作流程拆解

从创建项目文件夹开始，到初始化仓库、添加文件、提交变更，最后查看日志，完成 Git 项目的基础创建闭环。每一个命令都是版本管理的基础，构建起项目的历史记录链。



💡实用技巧：在实际开发中，若有多个文件或目录被修改，推荐使用 `git add .` 命令，可一次性将当前目录下所有新增、修改的文件添加到暂存区，简化操作步骤。

上传到 GitHub（首次连接）



01. GitHub 上新建仓库

登录 GitHub 账号，点击新建仓库按钮。请注意，不要勾选自动生成 README 文件，因为本地项目中可能已有相关文档，避免冲突。



02. 获取仓库远程地址

仓库创建完成后，在页面中找到 HTTPS 地址（格式如：<https://github.com/用户名/项目名.git>），复制该地址备用。



03. 本地终端执行关联命令

依次执行命令：关联远程库 `git remote add origin [地址]`；重命名分支 `git branch -M main`；首次推送并建立关联 `git push -u origin main`。



04. 后续更新，一键上传

首次连接成功后，后续对项目进行修改、提交（commit）后，只需在终端输入简单的 `git push` 命令，即可快速将本地最新代码同步到 GitHub 仓库。

克隆已有仓库（下载项目）

```
$ git clone https://github.com/用户名/仓库名.git  
$ cd 仓库名
```



执行以上命令后，Git 会将远程仓库完整地下载到本地电脑。进入项目目录后，你就可以像在本地项目中一样，自由地进行代码修改、提交和推送操作，无缝参与到团队协作或开源项目中。

日常最常用命令

git status

查看当前工作区文件的状态，了解哪些文件被修改、新增或删除。

git add .

将当前目录下所有修改的文件（新增、修改、删除）添加到暂存区。

git commit -m "说明"

将暂存区的内容提交到本地版本库，并记录本次提交的说明信息。

git push

将本地版本库的提交记录推送到远程仓库（如 GitHub），实现代码同步。

git pull

从远程仓库拉取最新的代码并合并到本地当前分支，保持代码最新。

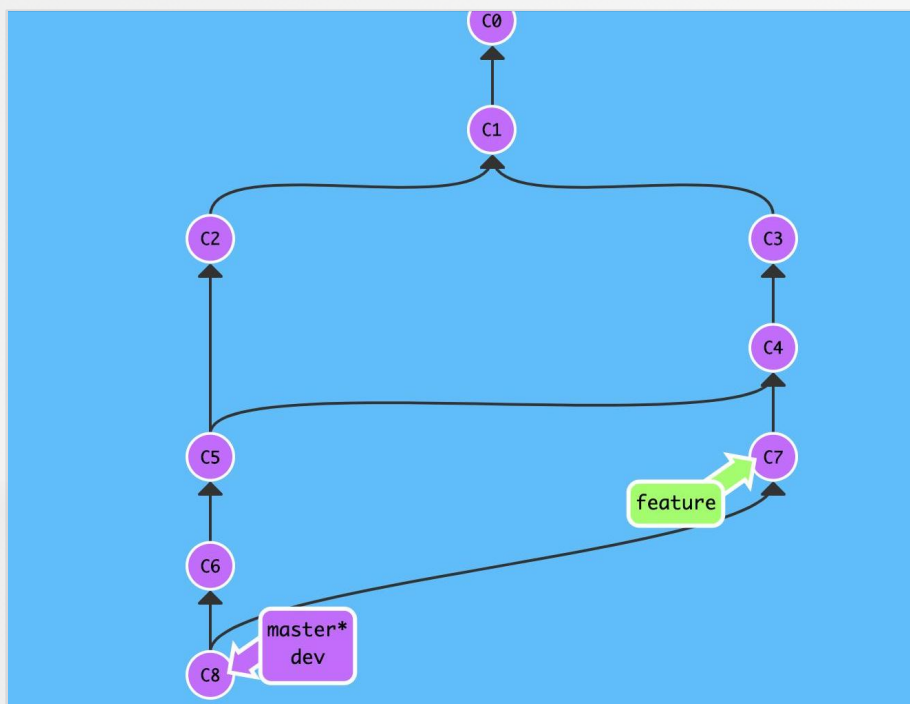
git log --oneline

以简洁的单行格式展示项目的提交历史，便于快速浏览版本变更记录。



推荐每次写代码前先执行git pull，拉取远程最新代码，避免后续合并时出现冲突，提升协作效率。

分支操作（平行版本）



分支是Git的核心功能，它允许开发者在不影响主线（main）的前提下，独立开发新功能或修复Bug，实现代码的“平行宇宙”管理。

创建并切换分支

新建 feature-login 分支并立即切换
`git switch -c feature-login`

查看本地分支列表

列出所有分支，星号(*)表示当前分支
`git branch`

切换回主分支

功能开发完成后，切回 main 分支
`git switch main`
准备合并

合并功能分支

`git merge feature-login`
将功能分支代码并入主线。

删除已合并分支

`git branch -d feature-login`
清理已完成任务的分支。

Pull Request (PR) 协作流程

PR 是团队协作的核心机制，本质是：“我改了代码 → 请你检查 → 没问题就合并”。它确保代码在进入主分支前经过评审，保障代码质量与安全性。



01. 准备工作

Clone项目代码到本地，基于主分支新建个人开发分支，避免直接修改主线。



02. 开发与提交

在分支上完成功能开发，规范Commit信息，将分支Push到远程代码仓库。



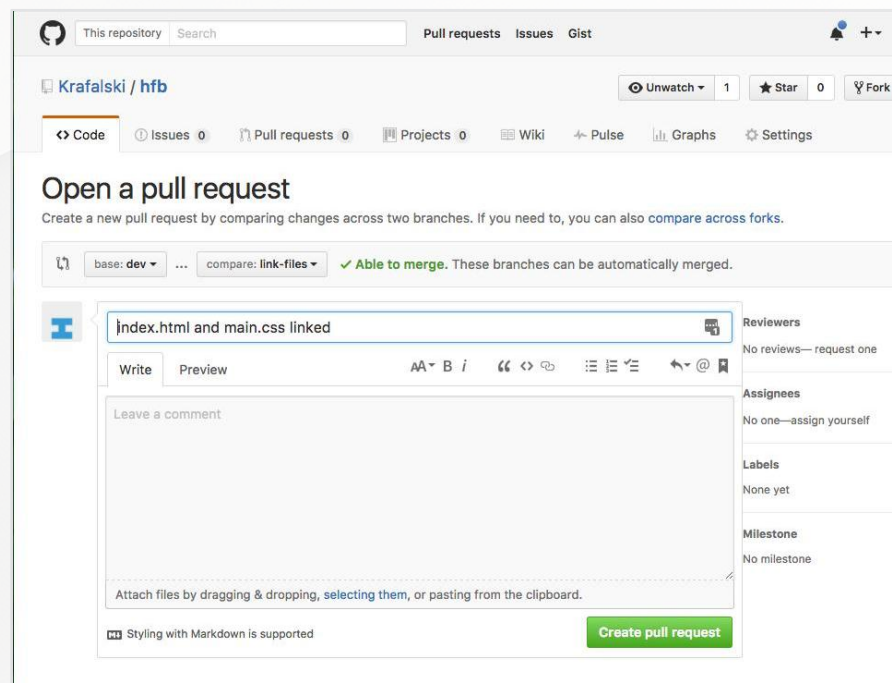
03. 创建与评审

在GitHub发起PR，邀请团队成员Review代码，讨论方案，迭代修改直至达标。



04. 合并上线

评审通过后，将功能分支合并到main分支，完成代码集成，形成完整闭环。



GitHub Pull Request 界面示意：清晰展示分支对比、变更文件与评审对话，是团队高效协作的核心载体。

SSH Key（免密推送）

01. 生成密钥对

在终端执行命令生成密钥，将“你的邮箱”替换为注册GitHub的邮箱，一路回车即可完成生成。

02. 查看并复制公钥

通过命令查看公钥内容，全选输出的文本并复制，这串字符将用于配置GitHub的信任凭证。

03. 配置GitHub密钥

进入GitHub个人设置，找到“SSH and GPG keys”，新建SSH key，粘贴刚才复制的公钥内容保存。

04. 测试与GitHub的连接

执行测试命令，若收到“Hi 用户名! You've successfully authenticated...”的提示，说明信任关系建立成功。

05. 切换仓库为SSH远程地址

将本地仓库的远程地址从HTTPS格式修改为SSH格式，替换命令中的用户名和仓库名，之后即可实现免密push/pull操作。

.gitignore – 忽略不需要上传的文件

在 Git 项目中，我们需要明确告诉版本控制系统哪些文件不需要被追踪和提交，例如自动生成的临时文件、环境配置文件或包含敏感信息的密钥文件，以保持代码库的整洁和安全。

```
__pycache__/ # 忽略 Python 缓存文件夹
.pyc # 忽略编译后的字节码文件
.env # 忽略环境变量配置文件
.venv/ # 忽略虚拟环境目录
```



安全警示： .env 文件中通常包含数据库密码、API Key 等敏感信息，务必将其添加到 .gitignore 中，严禁提交到公共代码仓库！

```
5  *.slo
6  *.lo
7  *.o
8  *.obj
9
10 # Precompiled Headers
11 *.gch
12 *.pch
13
14 # Compiled Dynamic libraries
15 *.so
16 *.dylib
17 *.dll
18
19 # Fortran module files
20 *.mod
21 *.smod
22
23 # Compiled Static libraries
24 *.lai
25 *.la
```

图示为代码编辑器中的 .gitignore 文件示例，通过清晰的规则列表，Git 能自动过滤掉这些指定的文件和目录，避免误操作。

常见错误及解决



git commit 后没有变化

这是最常见的疏忽，文件变更后需先执行 `git add`，将修改添加到暂存区，确保变更被 Git 追踪，之后再进行 `commit` 操作才能生效。



push 被拒绝 (rejected)

通常是因为本地代码版本落后于远程仓库。先运行 `git pull` 拉取最新代码，手动解决代码合并冲突后，再重新执行 `git push` 推送更新。



fatal: remote origin already exists

无需删除原远程库，直接使用命令 `git remote set-url origin <新仓库地址>` 即可修改已存在的远程仓库关联，快速更新项目的远程指向。



误提交了 API Key / 敏感信息

首要步骤是立即前往服务商平台重置密钥；随后用 `git rm --cached` 移除暂存区文件，将敏感文件加入 `.gitignore`，最后重新提交并推送代码。



重要提醒：API Key 一旦上传至 GitHub 等公开仓库，即使后续删除，也可能在提交历史中被检索到。因此，重置密钥是必须执行的安全操作，切勿抱有侥幸心理。

最小命令清单（先记住这些）

01. 起步与查看

git init（开始管理） | git status（查看工作区状态） | git log --oneline（查看历史提交记录）

02. 暂存与本地提交

git add .（将所有修改加入暂存区） | git commit -m "message"（提交到本地仓库，必须写说明）

03. 分支改名

git branch -M main（将默认分支名从master改为main，符合现代开源规范，初次上传前执行）

04. 关联远程与首推

git remote add origin <地址>（关联远程仓库） | git push -u origin main（首次推送并建立追踪）

05. 日常推送与拉取

git push（后续本地提交后，快速推送到远程） | git pull（从远程拉取最新代码更新本地）

06. 克隆已有项目

git clone <地址>（从远程仓库完整下载项目到本地，是参与开源或团队协作的第一步）



记忆口诀：init → status → add → commit → push / pull / clone，掌握这组流程应对80%日常场景。

Q&A

Thank You

从Git基础到GitHub协作，我们一同探索了版本控制的核心魅力。希望我分享的学习笔记能为你打开高效开发的大门，欢迎提出任何疑问，让我们共同探讨、共同进步。

